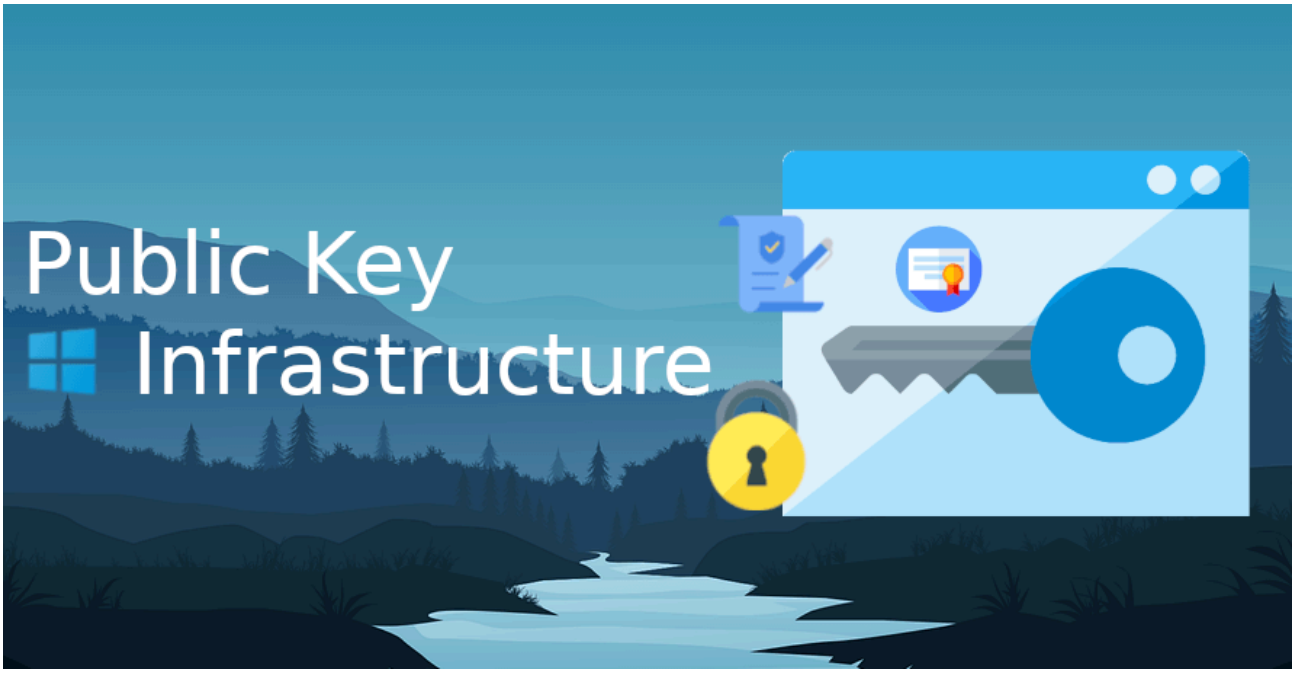
Top 10 PKI Recommendations by a former Microsoft Security Engineer – Michael Waterman

michaelwaterman.nl/2026/02/15/top-10-pki-recommendations-by-a-former-microsoft-security-engineer

Michael Waterman February 15, 2026



One of my recent posts about installing a two-tier Public Key Infrastructure did remarkably well, even got mentioned for the third time in the Microsoft Entra Newsletter! After publications I got many offline questions so I decided to do a follow-up blog on what's recommended when designing a PKI infrastructure, it's all stuck in my head anyway, so why not write it down. This post is not meant to be a theoretical PKI handbook. It is a practical overview of PKI best practices and common mistakes seen in real-world environments and a bit of my own experiences.

this list can be used as a baseline to validate whether your PKI aligns with industry-proven practices. This is also not an exhaustive list. It reflects the practices that matter most in day-to-day operations, and the ones that tend to go wrong most often. All based on my experience, hope it helps in your PKI endeavors.

1. Use a proper PKI hierarchy

For most organizations, a two-tier PKI is the recommended design:

- **Offline Root CA**
- **Online Issuing CA(s)**

A one-tier PKI (Root = Issuing CA) may work for very small or temporary environments but provides limited security. A three-tier PKI adds flexibility and policy separation but

increases complexity and operational overhead. Usually it's used only at very large distributed companies, the majority will need a two-tier.

Check out my blog posts on installing this kind of infrastructure. Security, flexibility, and manageability must be balanced deliberately.

2. Keep Root and Policy CAs offline

Root and Policy CAs should:

- Not be members of Active Directory
- Remain offline except for:
  - CA certificate renewals
  - CRL publication

Because offline CAs typically use long CRL validity periods (6–12 months):

- Delta CRLs should not be published
- The Freshest CRL extension should not be populated

Tip! *Configure DSConfigDN on Offline Certification Authorities. Offline CAs should have the DSConfigDN registry key configured. This allows the CA to locate the forest configuration partition when publishing CA certificates or CRLs to Active Directory. Even when LDAP is not used as an AIA or CDP repository, it is still recommended to publish Root and Policy CA certificates to Active Directory. This ensures they can be centrally distributed to domain members and avoids manual trust configuration. Explicitly configuring DSConfigDN prevents dependency on transient connectivity or assumptions during rare but critical operations such as CA certificate renewal or recovery scenarios.*

Keeping these CAs offline significantly reduces the risk of private key compromise. Rule of thumb, if a CA does not handout certificates on a regular bases, keep it offline.

3. Use strong (enough) cryptography

- If possible use Elliptic Curve Cryptography
- RSA keys must be 2048 bits or larger, prefer 4096
- SHA1 must *not* be used
- Use SHA256 for best compatibility
- SHA384 or SHA512 may be used where application support is guaranteed

Tip! *Ask your device vendors about the maximum RSA key length before making these decisions.*

Weak cryptography undermines the entire trust model of the PKI. If you want more information about the choices you have, see my blog about this specific topic.

4. Design CA validity periods correctly

A simple and effective rule:

> A child CA should have *half the lifetime of its parent CA*

Example:

- Root CA: 10 years
- Issuing CA: 5 years
- Leave certificates can not go beyond the remaining life time of the issuer

CA renewals should be planned, documented, and tested, especially renewals involving key rollover. I've got an entire blog written about this topic alone.

5. Design AIA and CDP locations correctly

Use HTTP, or AD integration if using domain joined devices

- SMB is not supported for AIA/CDP in modern Windows environments
- Preferably use HTTP-based repositories, it works on all operating systems
- Use DNS aliases or A records, not server namesExample: `trust.domain.com`
- When hosting the CRL on IIS, make sure it supports double escaping

This allows redirection and high availability without reissuing certificates and gives you a single form of management.

Use the correct variables

- **%4** is critical for CA certificates
- **%8** / **%9** are critical for CRLs

Without these variables, renewed CA certificates or CRLs may overwrite existing files, breaking validation paths.

Root CA certificates

- Should not contain AIA or CDP extensions, ever!
- *Root trust is established via the Trusted Root Store, not via revocation checking*

6. Use appropriate CRL validity periods

- **Offline CA**: 6 months to 1 year
- **Online Issuing CA**: maximum 7 days

Short CRL validity ensures revoked certificates are rejected within a reasonable time frame.

7. Harden certification authorities

Certification Authorities should be treated as tier-0 or high-value assets:

- No additional server roles
- Not installed on Domain Controllers
- Windows Firewall enabled
- Additional hardening such as CIS templates applied
- Use disk encryption where possible
- Do not, ever, use the web components of a PKI server. It’s obsolete and vulnerable.

Use the latest supported Windows Server version for all PKI components. There have been enhancements in the latest version, although small. Also remember that physical security is at play here. When you’re hosting the Root and Enterprise CAs on a hypervisor, make sure that the disks are encrypted. This prevents the person holding the admin keys to the hypervisor from stealing your data.

8. Enforce role separation and least privilege

Default CA permissions are overly broad. Best practice:

- Restrict CA administration to dedicated security groups
- Actively use role separation:
    - CA Administrator
    - Certificate Manager
    - Backup Operator
    - Auditor

Role	Primary Permission	Description
CA Administrator	Manage CA	Responsible for the overall configuration and maintenance of the Certification Authority. This role includes the ability to assign other CA roles, modify CA settings, and perform CA certificate renewals. Permissions are managed through the Certification Authority management console.
Certificate Manager	Issue and Manage Certificates	Handles operational certificate lifecycle tasks, such as approving enrollment requests and processing certificate revocations. This role is sometimes referred to as a CA Officer and is assigned via the Certification Authority management console. Local only.
Backup Operator	Back up files and directoriesRestore files and directories	Performs system-level backup and recovery of the CA. This role relies on operating system permissions rather than CA-specific rights and is essential for disaster recovery scenarios.
Auditor	Manage auditing and security log	Responsible for configuring, reviewing, and maintaining audit logs related to CA activity. Auditing is enforced at the operating system level and supports traceability and compliance requirements.

This reduces the blast radius of mistakes and insider threats.

9. Secure enrollment and certificate templates

- Limit Enroll and Autoenroll permissions strictly
- Avoid broad groups
- Templates that allow “Supply in the request” must require:
  Certificate Manager approval

This prevents abuse of Subject and SAN fields. See the addendum, or tip 11.

10. Plan for operations, monitoring, and recovery

Cert Publishers Group

In Active Directory integrated PKI deployments, the CA computer account must be a member of the *Cert Publishers* group in each domain where certificates are issued to users or computers. Membership in this group grants the Certification Authority the required permissions to write to the *userCertificate* attribute on user and computer objects. This is what allows issued certificates to be published back into Active Directory. From a functional perspective, this is easy to overlook. Certificate issuance may appear to work correctly, while certificates are silently not being published to AD. The impact typically surfaces later, for example during:

- certificate-based authentication scenarios,
- smart card or certificate logon,
- application lookups that rely on certificates stored in Active Directory.

From a security and operational standpoint, explicitly managing Cert Publishers membership also provides clarity. It makes the dependency between AD and AD CS visible and auditable, instead of implicitly relying on inherited or historical permissions. During assessments, I explicitly verify:

- whether the CA computer account is a member of Cert Publishers,
- in which domains this membership exists,
- and whether this aligns with the intended certificate issuance scope.

It is a small configuration detail, but one that directly affects both reliability and security of an AD-integrated PKI. Furthermore a production PKI must include:

- Regular backups (including private keys)
- Encrypted and physically secured backup storage
- Monitoring of:
    - CA services
    - CRL freshness
    - OCSP responders
    - Certificate expiration
    - Security events

Documentation is not optional. At minimum, document:

- Offline CA stand-up and retrieval
- Disaster recovery
- Emergency CRL signing
- CA renewals and rebuild procedures

Addendum – Tip 11: Use specialized tooling to assess ADCS security

When performing PKI and Active Directory Certificate Services (ADCS) assessments, manual reviews only get you so far. Configuration errors in ADCS are often subtle, difficult, and spreads across certificate templates, CA settings, and Active Directory permissions. Individually they may seem harmless, but combined they can significantly weaken the security posture of the environment.

One tool I consistently use during security assessments is Locksmith, created by Jake Hildreth. Check out his GitHub here. Locksmith is specifically designed to analyze ADCS deployments from an attacker's perspective. It highlights mis-configurations that can lead to:

- certificate-based privilege escalation,
- abuse of overly permissive certificate templates,
- dangerous enrollment configurations,
- and trust relationships that are often overlooked during day-to-day operations.

What makes this tool particularly valuable is that it bridges the gap between theory and reality. Many of the issues it detects technically "work as designed", but introduce risk because of how they interact with Active Directory and modern attack techniques. These are exactly the kinds of weaknesses that are easy to miss when focusing solely on best-practice documentation or individual settings.

In several assessments, Locksmith provided immediate clarity on *why* a PKI felt uncomfortable from a security perspective, even when it appeared healthy on the surface. It does not replace architectural review or operational validation, but it significantly accelerates the process of identifying where deeper analysis is required.

As with any tooling, the output should be interpreted in context. Not every finding is automatically critical. However, using a tool like this as part of a broader PKI assessment helps move discussions away from assumptions and towards concrete, evidence-based decisions. Go ahead and give it a try!

Closing thoughts

During my time working with customers as a Microsoft Security Engineer (PFE), I rarely encountered PKI environments that failed because of weak cryptography choices alone. In most cases, the cryptographic primitives were technically sound, key sizes were sufficient, SHA2 was sometimes already in use, and the platform itself was capable enough. Where things usually went wrong was elsewhere.

Design decisions were often made early on for convenience and never revisited. Offline CAs were brought online more often than intended. AIA and CDP locations grew organically instead of being designed. Certificate templates accumulated permissions over time and never checked again. Documentation existed at some point, but no longer reflected reality. And operational tasks such as CRL publishing or CA renewals were treated as “something we’ll figure out when we get there”.

None of these issues are dramatic on their own. But PKI is an ecosystem where small shortcuts compound over time. When something eventually breaks, an expired CA certificate, a missing CRL, an unavailable AIA endpoint, it usually happens under pressure, without clear procedures, and with very little room for error.

That is why I tend to look at PKI less as a cryptographic system and more as a long-running operational service. One that will outlive projects, administrators, and sometimes even entire infrastructure platforms. If it is not designed with that in mind from the start, it will slowly drift into a state where it still “works”, but no one fully understands it anymore…. does anyone have the documentation around?

The practices outlined in this post are not theoretical ideals. They are patterns that repeatedly proved their value in real environments, often after something had already gone wrong, and we would be called on-site. Used early, they help prevent that situation. Used later, they provide a structured way to regain control and confidence in an existing PKI.

If this list helps you ask better questions about your own PKI, or validates design choices you already made, then it has done its job.

Use these practices as:

- a design checklist,
- an assessment baseline,
- or a maturity benchmark.

I hope this information is useful for you, if you have any question, please just let me know!